Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Artificial Intelligence
Agentic AI (IA462), Semester VIII

U23AI052 LAB 7 SUBMISSION

Experiment 7

Agent Behaviour as a Markov Decision Process — Value and Policy Iteration

Aim

To model an agent acting in a stochastic environment as a Markov decision process, compute the optimal value function and policy by value iteration, obtain the same policy by policy iteration from an arbitrary starting policy, and compare the two methods by the work each performs.

Theory

An MDP states an agent's problem as five things: the states it can occupy, the actions available in each, a transition function giving the probability of landing in each next state, a reward attached to the outcome, and a discount factor weighting later reward against sooner. Because the next state depends only on the current state and action, a solution need not be a sequence of moves but a policy — one action per state — and the value of a state under a policy is the expected discounted return from it. The Bellman optimality equation says the optimal value of a state is the best, over actions, of the expected reward plus the discounted optimal value of where that action leads.

Two algorithms solve it. Value iteration treats the Bellman equation as an update rule and applies it to every state until the values stop changing, reading the policy off at the end. Policy iteration alternates instead between evaluating the current policy exactly and replacing it with the policy greedy in those values; it converges in very few improvement rounds but pays for each with a full evaluation. The distinction worth carrying into the rest of the course is that here the transition and reward functions are given, so the agent plans rather than learns — this is the model that Experiment 8 removes when the agent must estimate the same quantities from experience.

Table 1. The environment to be modelled.

Element	Specification
States	the cells of a 4 by 3 grid; the cell at column 1, row 1 is a wall and is not a state
Actions	up, down, left and right in every non-terminal cell
Transitions	the intended direction with probability $1 - 2p$ and each perpendicular direction with probability p; a move into a wall or off the grid leaves the agent where it is
Rewards	$+1$ on entering the terminal at column 3, row 2; -1 on entering the terminal at column 3, row 1; a fixed step reward in every other cell
Parameters	slip probability $p = 0.1$, step reward -0.04, discount factor 0.95, start at column 0, row 0

Algorithm

Algorithm 1 — Value iteration

1. Build the MDP of Table 1: enumerate the states, and write a transition function returning the list of probability and next-state pairs for each state and action, folding the cases where a slip and the intended move lead to the same cell. Verify that the probabilities out of every state and action sum to one.

2. Initialise all values to zero and sweep repeatedly over the non-terminal states, replacing each value by the best expected reward plus discounted value over the four actions, until the largest change in a sweep falls below a stated tolerance. Print the converged values laid out as the grid, the policy greedy in them, the number of sweeps and the value of the start state.

```
(powermind_rtx5050) D:\Agentic AI lab>python lab7/lab7.py

=== algo1: value iteration ===
values
14.974 16.133 17.248 17.479
14.000 wall 16.133 17.141
13.191 14.010 14.984 15.075
policy
R R R U
U wall U U
R R U U
sweeps: 448 start: 13.190736
```

Algorithm 2 — Policy iteration

1. Start from an arbitrary policy — for instance up everywhere — and evaluate it by sweeping the Bellman update for that one action per state until the values settle, which gives the value of acting under that policy rather than optimally.
2. Replace the policy by the one greedy in those values and repeat the evaluation; stop when an improvement round leaves the policy unchanged. Print the final policy, the number of improvement rounds, the total evaluation sweeps and the number of value updates performed, and check that the policy and the values agree with Algorithm 1.

```
=== algo2: policy iteration ===
values
14.974 16.133 17.248 17.479
14.000 wall 16.133 17.141
13.191 14.010 14.984 15.075
policy
R R R U
U wall U U
R R U U
improvement rounds: 4 evaluation sweeps: 1762 value updates: 19382 start: 13.190736
```

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the program below, including the converged value grid and policy grid from value iteration, the same two from policy iteration, the sweep and update counts for both, and the value of the start state under each.

Observations

Comment on why the policy sends the agent the long way round rather than along the row holding the negative terminal, on what the value of the start state means in units of reward, on which method reaches the answer in fewer sweeps and which performs fewer value updates in total, and on what the discount factor contributes when every episode ends at a terminal anyway.

Exercises

1. Sweep the step reward over −0.01, −0.04, −0.1, −0.5 and −2.0, and separately sweep the discount factor over 0.5, 0.7, 0.9, 0.95 and 0.99, reporting for each setting the policy grid, the value of the start state and the sweeps to convergence. Identify every setting at which the policy changes and state in one sentence what the agent is now preferring.

```
comparison: same policy = True same values = True

=== ex1: step reward and discount sweeps ===
step reward: reward, policy, start value, sweeps
-0.01 URUUUUURRRU 13.3774 448
-0.04 RRUUUUURRRU 13.1907 448
-0.1 RRUUUUURRRU 12.8199 448
-0.5 RRUUUUURRRU 10.3479 446
-2.0 RRRUURURRRR 3.0945 444
discount: discount, policy, start value, sweeps
0.5 URULUUURRRU -0.0125 35
0.7 URUUUUURRRU 0.3895 66
0.9 URUUUUURRRU 4.9133 219
0.95 RRUUUUURRRU 13.1907 448
0.99 RRUUUUURRRU 82.5452 2279
```

2. Sweep the slip probability over 0, 0.05, 0.1, 0.2 and 0.3, reporting the policy, the value of the start state and the probability that an agent following that policy from the start state ever reaches the negative terminal. Explain why that probability does not rise steadily with the slip probability.

```
=== ex2: slip probability sweep ===
slip, policy, start value, negative-terminal probability
0 URUUUUURRRU 16.1417 0.0
0.05 RRUUUUURRRU 14.7772 0.000145
0.1 RRUUUUURRRU 13.1907 0.001355
0.2 RRUUUUURRRU 9.2409 0.014706
0.3 URRUUUUURRU 4.4282 0.066502
```

o